



UiA Research Dashboard
Bachelor Thesis in DAT304

Group 9

Filmon Berhe Mebrahtom, Mohamed Ali Abdullahi And Yeronis Assefa Hubena

Supervised By Professor Arne Wiklund

Faculty of Engineering and Science,
University of Agder

Grimstad, January 2021

Abstract

This Bachelor thesis involves developing publication dashboard for the University of Agder (UiA). The aim of this project is making research and academic publications more visible and store data for further analysis and visualisation purposes. There are different Fields, faculties and departments in the University of Agder(UiA). Several researches and publications have been accomplished so far. On the main website of the university, a summary of the various research projects that UiA has completed or is currently working on is briefly displayed. However, it is not as organized and inclusive of every accomplishment the school has achieved. Hence, For the Bachelor Thesis in Computer Engineering, Our group has developed a responsive and easy to use web application. The website displays charts and graphs of the researches published, according to faculties, institutes, research groups and other institutions UiA have collaborated with. Additionally, it has authorization and authentication functionality for the users and administrator. In the website there is a tutorial video presentation that helps users to understand how the website functions.

The group had meetings throughout the development process and notes were taken during these meetings. Each member of the group had their own role such as, group leader, meeting referral and notes taker and daily topic discussion presenter.

The front-end part of the project used a single page application with React JavaScript framework, axios, fetch API and other libraries to help develop the website. The back-end was developed using ASP.NET core and communicates with both the front-end and CRISTIN API. It fetches data from CRISTIN and store it in a local database and send it to the front-end.

It was quite an experience working on this project. Our plan for the development has been achieved and we have learnt so much, as a group and independently about what it means to develop and accomplish a product.

Preface

We would like to thank our families and friends for all the support they have offered during the developmental process of this project. Special thanks to our supervisor Professor Arne Wiklund for giving us good advises and suggestions. Thanks for everyone who took their time to help answer our questions.

University of Agder, Grimstad, 14.May.2021

Yeronis Assefa Hubena, Mohammed Ali Abdulahi, Filmon Berhe Mebrathom

Contents

1	Introduction	1
1.1	Report Structure	1
1.2	Thesis Definition	1
1.2.1	In-Scope	2
1.2.2	Out Of Scope	2
1.2.3	Assumptions And Constraints	3
1.2.4	Project Organization	3
1.3	Theoretical Background	4
1.3.1	Academic Publication	4
1.3.2	Data visualisation and Dashboards	5
1.4	Technical Background	5
1.4.1	Web Services and Web Application program Interfaces(APIs)	5
1.4.2	Client - Server Architectural Pattern	6
1.4.3	Front End Framework - Single Page Application	7
1.4.4	React	7
1.4.5	Entity relationship And Entity Framework(EF)	8
1.5	Tools	8
1.5.1	Jira	8
1.5.2	Rider	9
1.5.3	BitBucket	9
1.5.4	LaTeX- Overleaf	9
2	State Of The Art	11
2.1	Aura (UiA.brag.unit.no)	11
3	Method	12
3.1	Process	13
3.2	development tools	13
3.2.1	Database related tools	14
3.2.2	Axios	14
3.3	Postman	15

3.4	JSON Web Token (JWT)	15
4	Solution	16
4.1	Overview	16
4.2	Non-Functional Requirements	16
4.2.1	Modifiability	16
4.2.2	Usability	17
4.2.3	Performance	17
4.3	Database	17
4.3.1	Database design	17
4.3.2	Fetching Data and Loading To Database	18
4.3.3	Database Synchronisation With Remote Database(CRISTIN)	21
4.3.4	Background tasks	21
4.4	Frontend Architecture	22
4.4.1	Components	22
4.5	Testing	23
4.5.1	Postman - API-backend tests	23
4.5.2	Testing Amount of Publication Per Institutes	24
5	Result	26
5.1	High level System Architecture	26
5.2	Database	27
5.3	Initially Populating the Database	30
5.3.1	Caching	32
5.4	Application	32
5.4.1	Home page	32
5.4.2	Registration	34
5.4.3	Group Research	34
5.4.4	Faculties and institutes	35
6	Discussion	36
6.1	Results	36
6.2	Process	36

6.3	Challenges	36
6.3.1	Choosing best filter Parameters	37
6.3.2	Authentication and Authorisation at the database level	37
6.4	Deployment challenge	38
6.5	Future Work	38
7	Conclusion	39
	Appendix A Group Declaration	44
	Appendix B Group Contract	44
	Appendix C Group Meeting Discussion	44
	Appendix D Preliminary Project	44
	Appendix E Querying CRISTIN for publication from PostMan	44
	Appendix F Dashboard Database	44
	Appendix G Publication per institute	44

List of Figures

1	Client Server Architectural Pattern	7
2	Development Activities	12
3	Overview	16
4	Simple graphical presentation of Data Fetch and Load to database	19
5	Entity Framework managed mapping (ORM)	20
6	Synchronisation With Remote Database	21
7	Component architecture	22
8	Automated testing	24
9	Illustration of Headers In Post Man	25
10	System Architecture	26
11	Database for Dashboard	28
12	Database Designed in MySQL Workbench	29
13	Publication Per Institutes Since 2017:- table populated 19 April 2021	31
14	Home page	33
15	Home page	34
16	Top Ten Group Research	35

List of Listings

1 Introduction

This thesis, explores the development of a publications dashboard for the University of Agder. Three central concepts define this report; Academic research, Data visualisation and visualisation Software(Dashboard). Introductory definitions, concepts and technologies related to those three central elements of the project are given in this chapter. This chapter introduces the reader to the underlying theories and technologies in this report. Some of the contents at the theoretical background that can be found at the subchapter 1.3 explains the concepts of academic research and the Norwegian model to measure research productivity. Technical backgrounds are mostly explained in 1.4.

1.1 Report Structure

This report is divided into seven chapters. Each chapter of the report goes into detail about our product. The report's introduction section provides readers with an overview of the Publication Dashboard. In the State of the Art section, we discussed some of the products that are similar to ours and are currently available on the market. The Method section describes the process and tools that were used to develop this project. Other chapters, such as solution and result, detail the approaches we took to resolve issues, construct the product correctly, and, as a result, the outcomes we discovered after the product was constructed. Additionally, in the discussion section, we discuss the outcome, the challenges we encountered, how we can continue to develop and expand our product in the future, and the entire developmental process. Finally, the conclusion section summarizes the report.

1.2 Thesis Definition

The Norwegian Higher education institution and research sectors publish their research papers to the national research information system in Norway (CRISTIN)[25]. The University of Agder, located at the southern region of Norway is one of the higher education institutions that report and register research activities to CRISTIN. Given that CRISTIN is a nationally shared database, this project aims to develop dashboard with onsite database for the local publication activities at the University of Agder.

The dashboard shows an overview of research activities at the university. The system includes

local database, and the data will be imported from CRISTIN. Storing a copy of the needed data locally can also help with off-loading work from CRISTIN, in addition to the benefits for the University of Agder.

1.2.1 In-Scope

Main features and functionalities of the software include, fetching projects and publications data registered by UiA from Cristin, map the data to target data-models, store in local database, group data categorically and visualize the data using graphs and diagrams. CRISTIN does not necessarily provide the desired structure or all desired information. Therefore, the system have to restructure the received data, to meet the desired result. Additional information necessary for the establishment of relationship between data models, can be fetched from the University's web page. The database periodically synchronizes with the remote data sources, in this case CRISTIN.

The User Interface include displaying aggregated data according to some metrics. The aggregation metrics include faculties, institutes and publication year among others. The data can also be presented in different type of charts. **Summary of In-scope as list**

- Gather information about research groups, their members, collaborators, publications and design a data model.
- Periodically fetch publication data from CRISTIN.
- Save the data into the local database.
- Aggregate data.
- Dashboard that visualise the aggregated data.

1.2.2 Out Of Scope

Registering and reporting of publications is not part of the responsibilities of the system, as reporting and registration is done at CRISTIN. This system, however, updates its database regularly by frequently requesting and fetching data from data sources, thus displaying information updated periodically. The system, furthermore, implements user authentication and authorization.

1.2.3 Assumptions And Constraints

Assumptions about the project

- The Software is able to run on all platforms such as, Windows, Mac OS or Linux, on a computer.
- The software is a web Application.

Constraints about the project

- Use Cristin's API to access and fetch data.
- This software serves only the University of Agder, as the main purpose of the project is to graphically present research activities at the University of Agder.
- The Cristin API sends the data in JSON file form, thus the program we are going to create must be able to receive Json file format.

1.2.4 Project Organization

The team is composed of three members. Each member automatically assumes a developer role. In addition to programming each member does research and write report. Furthermore one member took a leader role, the second a note taker and the third discussion presenter before the group.

1.3 Theoretical Background

1.3.1 Academic Publication

Any research endeavors to generate new knowledge, and publishing it through established publication channels is critical for disseminating this new knowledge to the scientific community[40]. Publishing thus can be an indicator to the amount of knowledge production, and citations can indicate the influence of the research on the field[40].

Through research, experts enrich their own knowledge and contribute to the advancement of their field of study. Gathering information, analysing it and probably discovering new ideas, does not however guarantee wide spread acceptance of the finding. To make any impact on the area of study, to be accepted and be utilised by other professionals, the research result should be submitted to approved publishing channels, where in addition to other processes, it is peer-reviewed by researchers from the field and validated.

Millions of scientific articles pass through the above process, and become published. Evidently, validating the work does accomplish something but standing out from the millions of articles needs more than publishing it. Publications should be accessible to as many researchers as possible and let others build up on the result. During this era of daily generation and addition of huge amount of data to the internet, visibility becomes important factor for research articles to make an impact.

In recent years, the need to measure the result and effects of researches and innovations has increased, both in Norway and globally[3]. A part of the public funding for Norwegian higher education is contingent upon their productivity, which is measured in the form of scientific publications[36]. The Norwegian ministry of education introduced publication indicator model in 2006 and revised it in 2015[36]. The Current Research Information System In Norway (Cristin)[25] is established to serve institutions as shared database for reporting publications to the ministry of education[25].

This document reports the development of publication dashboard for the University of Agder. The Dashboard's main goal is to report the amount of research activities and knowledge production at the University, using simple graphical representations. The source of the publication data the software collects is Cristin and the fetched data is to be stored locally.

1.3.2 Data visualisation and Dashboards

Data visualisation is a vast field and it is a technique to communicate data in a more understandable manner using visual encoding, such as charts, diagrams, tables and other graphical representations. The technology reports data in a way that gives a quick insight into the data and helps in decision making.

The process of data visualisation generally involves 7 steps, however, projects can choose to follow all steps or some of them[14].

1. Acquire: Gather data or obtain it from a data source.
2. Parses: Structure the data in categories.
3. Filter: Remove redundant information.
4. Mine: Apply mathematical or statistical to look for patterns.
5. Represent: Choose a graphical representation model such as table, chart or graph.
6. Refine: Improve the graphical representation to make it more engaging.
7. Interact: provide methods for manipulating the features of presentation.[14]

Data visualisation technology can be broadly categorised in two technologies: Visual reporting and Visual analysis[9]. Visual reporting is mainly dashboard or scoreboard, which shows performance of a business using charts or graphs. Visual analysis, on the other hand, goes further, by encompassing statistical, and predictive analytics [9].

1.4 Technical Background

1.4.1 Web Services and Web Application program Interfaces(APIs)

Web services are technologies that enable distinct applications to exchange data or integrate with one another over a network, regardless of language, platform and/or Framework[26]. They can be considered as an approach to expose functionalities of an information system and make it accessible through standard web technologies.

Web services embody a distributed computer infrastructure, comprised of different applications or components, that interact and communicate over a network to function as a single logical

system [26]. Web services can differ in function from a simple request, like fetching data from a database, to complete applications that process data from multiple sources[26]. Standardised protocols and styles of web application programming interfaces(API) act as a glue between services and applications. Popular among these are SOAP(Simple Object Access Protocol) and REST (Representational State Transfer), while GraphQL, a data query language and specification released by Facebook in 2015[37], is gaining popularity.

Even though SOAP, REST and GraphQL are widely used to create API architecture, they have major differences. SOAP is a protocol, and uses XML(extensible markup language) as its data format[31]. REST Stands for Representational State Transfer and it is an architectural style[13] that allows different data representation formats including XML and Json(Javascript Object Notation)[30]. GraphQL in the other hand, is a data query language, and one of its main appeals is that it can fetch all data in one request instead of sending multiple requests[15].

1.4.2 Client - Server Architectural Pattern

Distributed system architecture divides a system into components that communicate over a communication network[4]. Client - Server pattern is one of many distributed system architectural patterns, that divide the responsibilities of the system between system components. Client - Server pattern as its name implies consists two components: Client and Server. The client request services or resources from the server and the server provides the services or resources.

Since the client - server pattern is based on service requester and service provider, servers can in turn be clients of other servers, if they request services or resources from other servers. Figure [1] summarises the client - server architectural pattern.

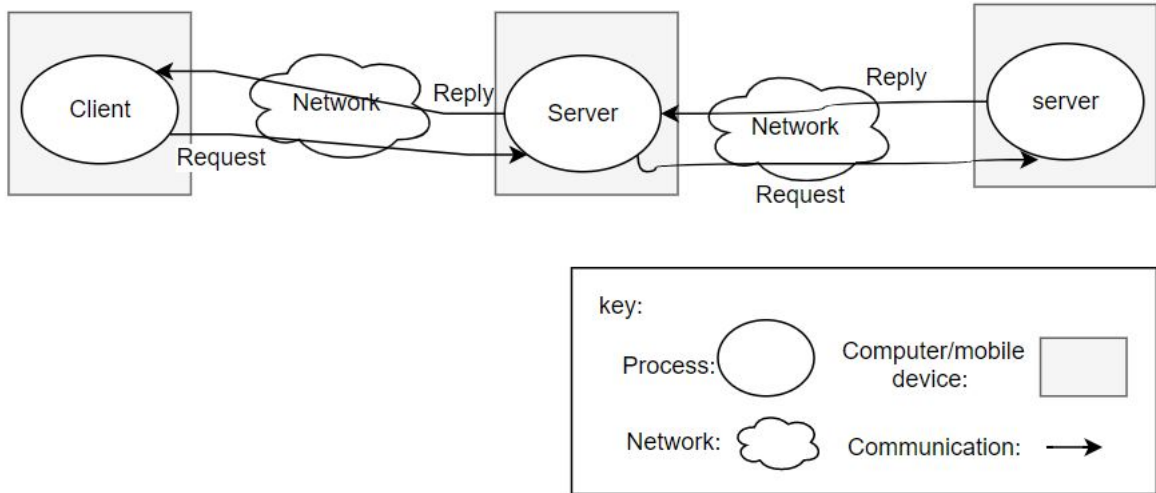


Figure 1: Client Server Architectural Pattern

1.4.3 Front End Framework - Single Page Application

"A single page application (SPA) is a web application that runs a single HTML page. It updates the page dynamically, rather than loading a new page"[41]. It provides good interaction between the user and the application itself. Unlike multi-page application, which involves multiple, separate pages, navigation in a single page application does not require visiting an entirely new page. The page, instead, runs within the same page. Single page applications include social media applications such as, Facebook, Whatsapp and Instagram. Another example of application that uses single page application is Google Maps. It is used to display a real-time view changes. Real-time updates are such as, notifications, real-time charts, and data streaming, where it is necessary to display a constant update of the current situation. React Js, Angular, and Vue Js are some examples of a single page application framework, which are widely used JavaScript libraries. In this project, React Js library was used to build the front-end application [19].

1.4.4 React

React is one of the popular JavaScript Library for building user interfaces. "It is a declarative, efficient, and flexible JavaScript library for building user interfaces"[38]. React Components, are used to implement a render method, which receives an input data and returns what's needed to

display. These input data can be accessed by the `render()` method by using the `"this.props"` parameters. Components can maintain data that are accessed through `this.state` method. Therefore, as the component state changes, the page will be updated by calling the `render()` method. When a user writes a URL in the search engine bar and click enter, he/she expects to be directed to the page they want to access. In order to execute this, one can use techniques such as Routing. Routing is a process of mapping URL's to their correct destination by using different JavaScript libraries such as React Router. The React router library has the capability to provide routing to single page applications that are built using react. Hence, this will allow the users to access the correct web page [29].

1.4.5 Entity relationship And Entity Framework(EF)

An entity is a distinct "object" which can be differentiated from other objects by the values of its properties.[30]. Entities with the same type of attributes belong to an entity set, and a relationship is an association between two or more entities. In relational database approach, entity sets are stored as tables, whose rows representing individual object entries, and columns representing entity attribute.

In object Oriented programming languages (OOP), data is modeled as objects, for example C# models. According to[30] Object-oriented databases, are developed to store data modelled in object-oriented programming language, but were not successful. Today object-oriented programming languages such as, C# is storing objects in relational database, with the help of Object relational mapping techniques and frameworks.

Entity framework(EF) is an Object-relational mapping framework (ORM), developed by Microsoft and is a tool that simplifies mapping between objects in objectoriented programming languages and tables in relational database[12]. Entity Framework is part of the .Net Framework[1] and helps translate between object oriented data models and the data format relational database understands.

1.5 Tools

1.5.1 Jira

Jira is one of the most known project management software platforms currently. It is used to manage projects efficiently and make our work easier. It is presented in three different packages.

These are, Jira core, which is the basic project management platform. Jira software, has an Agile functionality in addition to acquiring jira core's functions and Jira service desk. This software is used in several tasks, where project management is vital such as, bugs and problems tracking, kanban and Scrum managements to track backlogs, sprints and timesheets in a very organized form and so forth. It is a good tool to keep the team on track. [20]

1.5.2 Rider

Rider is a JetBrains cross-platform development environment for developing .NET applications on a variety of operating systems, including Windows, Mac OS, and Linux. It lets us edit, debug, build and run our codes in a fast and organized manner. It has features such as, code analysis, which fixes code issues or let us know if there is any problem in the code. Code editing, refactoring, unit test runner, Debugger, database, Navigation, Front-End technologies, version control and so much more, are some of the several features of Rider IDE. [8]

1.5.3 BitBucket

Bitbucket is a git code management that lets the team collaborate on their tasks and codes. It helps them to plan projects, test the project and deploy it. It provides features such as access control, where it allows members with permission only, can access the source code. Jira integration is another feature of Bitbucket, which helps with complete task management. It helps the team to integrate and review each other's code via pull request.

For Bitbucket, there are three different options of deployment. Bitbucket Cloud is accessed through a URL and deployed on Atlassian servers. It includes a tool called pipeline, that lets us build, test and deploy straightaway from bitbucket. Bitbucket Server does not have a tool for testing and deploying but it integrates with Bamboo, a tool that allows us to automate the building process. Bitbucket Data Center is hosted in a cluster of several servers in our environment. It provides benefits such as high performance, high availability and, smart mirroring. [42]

1.5.4 LaTeX- Overleaf

Latex editor is a system that lets a team to collaborate and work on a document together. It is a very organized and detailed method of editing a document. Overleaf is a latex editor with several features. It has a content of the document on one side of the screen and a PDF overview

of the output of what's being written. This editor is able to highlight, edit, insert tables, symbols, images, charts, it also provides an organized method of referencing sources and various other features. Generally, it is a way of including all the documentation process, the writing, reviewing and publication, together.[11]

2 State Of The Art

2.1 Aura (UiA.brag.unit.no)

One local system that shows the overview of the amount of publications done at the University of Agder is Aura[5], a service from Unit[39]. Aura is a local archive for academic research, dissertation, and thesis at the university of Agder[5]. Aura is based on a full-stack web application software named Dspace[35], Dspace is an open source software[6] and its architecture includes a front-end, storage manager and a database[35].

The similarity of Aura with the system we are building is that, aura imports publications from Cristin[28] and shows the amount of publications imported from CRISTIN based on Faculties, Departments, year of publication and more. Aura presents them in a list format, showing the numbers in square brackets.

3 Method

As mentioned at the introduction section, the primary operations of the software are fetch, Map, group and visualise operations. The system is to be built from scratch, therefore the development process, as Fig 2 shows encompasses six main activities; Requirement definition, designing, implementing, integrating and testing, and deployment.

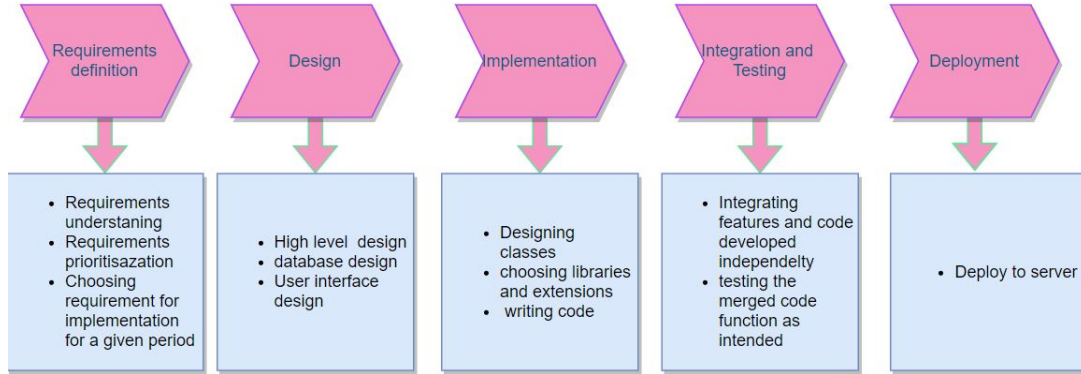


Figure 2: Development Activities

At the beginning of the project, two well known development models, waterfall and Agile approaches, seemed right for the assignment as discussed by the team. Waterfall model is a sequential process, where all the process activities and schedule are planned in advance[32]. The general principle of waterfall model, is completion of one phase before starting another phase in the development life cycle. In contrast to the waterfall model, the Agile model incrementally develops a software, but delivers a useful working software in each increment. In the agile model all or some of the development activities are part of each increment, and this enables the process to handle changing requirements, or integrating new technologies during the project life cycle[32].

In the beginning, the team considered the waterfall model, however, it quickly became apparent, that developing the project sequentially was not feasible. The reasoning is simply because the assignment came with general description which implied that requirements could change during the development period. In Addition, the team lacked prior knowledge of how the research activities work and had limited technical experience for such a system, which re-enforced the reasoning. These factor left no choice but building incrementally, which has the "flexibility" advantage in managing unexpected changes during the development period.

3.1 Process

Under this chosen model the first five of the activities mentioned at the first paragraph of section 3 are wrapped into a single iteration. The Duration of an iteration varies from two to three weeks.

1. **Requirement analysis:** Requirements are gathered during this phase. During this stage the team analyse the requirements and consult with assignment owner to develop mutual understanding of the expected functionality.
2. **Design:** This phase gets input from the requirement analysis, and use the information to design data models other software elements and their relations.
3. **Implementation:** The design Phase sets the stage for the actual building of the system. Developers are assigned tasks and start implementing them. This activity adds working components/features to the product.
4. **Integrating and Testing** Usually functionalities are implemented separately by the responsible developers. To function as a single software capable of performing desired business logic, code integration and testing is needed. This ensures the independently developed code operate as a single software, and delivers the intended service.

The System comprises three logically independent components: a client app , a web application services, and background services that fetches data from remote data source and save to the local database. Each developer took responsibility for one of the components. The mentioned structure provided the team members flexibility and independence when choosing tasks, so long as the needed prerequisites are in place and the task satisfies the agreed-upon priorities. The group often held meetings to resolve programming issues and minimise possible misunderstandings regarding the problem space. As a result, all refining requirements, some of the design, some code merging and testing were executed together during these sessions and meetings.

3.2 development tools

Tools that were utilised to build Publication Dashboard in UiA are discussed in this sub section.

3.2.1 Database related tools

MySQL Workbench/MySQL Server

MySQL is a relational database management system and MySQL Workbench is software that provides tools for model designing, SQL¹ development and server administration[23]. MySQL workbench were the primary database designing tool for the Publication dashboard, as well as migrating the database scheme to MySQL server and writing sql scripts during the very beginning of the development period, when publication Dashboard lacked necessary components to test the database, MySQL server were used to test the database model.

SQLite

SQLite is an open-source SQL database-engine, that were used by the software during the development stage. SQLite is server-less engine that writes directly to disk files, and is most widely deployed engine[34] . The decision to deploy SQLite was based on the fact that SQLite does not require a separate server, and can be easily installed directly from JetBrains Rider IDE².

3.2.2 Axios

React applications has the ability to display data fetched from external APIs, in the web page. In order to retrieve this data, methods such as Fetch API or Axios can be used. Axios uses the HTTP GET and HTTP POST requests to retrieve external data. "Axios is a promise-based library, which gives you the ability to take advantage of JavaScript's async and await for more readable asynchronous code"[16]. When we say promise-based library, we are referring to the object that represents a complete successful or failed asynchronous task. A Promise can have three different states, such as, pending state, which is the first state before it completes successfully or fails to operate the task. The fulfilled promise state is where the task performs a successful completion of the task with the expected value and the rejected state is when the task fails with an error.[2]

¹Structured Query Language [33]

²Integrated Development Environment

3.3 Postman

Postman is an API client that simplifies the process of developing, testing, and documenting APIs. It enables the development and storing of both simple and complex HTTP requests, as well as the reading of their responses[17]. Each Postman request includes a header and authentication token, but these tokens can be customized. By using postman, tests can be performed and results can be saved for later use. In addition to this, it is possible to make a group requests to the APIs.

3.4 JSON Web Token (JWT)

When a user logs in successfully using their credentials, a JWT is created and returned to the client. JSON Web Token(JWT) is a token that the user uses to get access of the protected resources on the server. Tokens can be stored locally, either in local or session storage. When a client tries to access a protected route or resource on the server, the token should include 'HTTP request', in order for the server to authorize the client. Since the data is digitally signed, it is trustworthy and safe to access. Both the client and server can safely transfer data using JWT tokens. By using public/private key pairs, we can verify that the senders are indeed who they claim to be. Additionally, since the signature is determined using the header and payload, it serves as a check to ensure that the content has not been altered[21].

4 Solution

4.1 Overview

The website is built specifically for University of Agder (UiA) and will be using the university's server. The front-end part of this website is a single page application (SPA) and uses React JavaScript library to build user interface. The website displays different types of charts to show the data on researches made in UiA, fetched from the CRISTIN API. The back-end retrieves data from the API and store it in the local database. This helps the front-end to fetch the data stored in the back-end and perform the necessary operation. The front-end displays charts of the data retrieved through the JavaScript services. It includes features such as granting access to the superadmin to register new members of the research group and checkboxes that allow users to choose between the university's various faculties and display charts of the faculty's institutes. React JavaScript and the page components serve as the user's interface, meaning, they send requests to the back-end while the controller serve as interface for obtaining the request and respond back to the front-end.

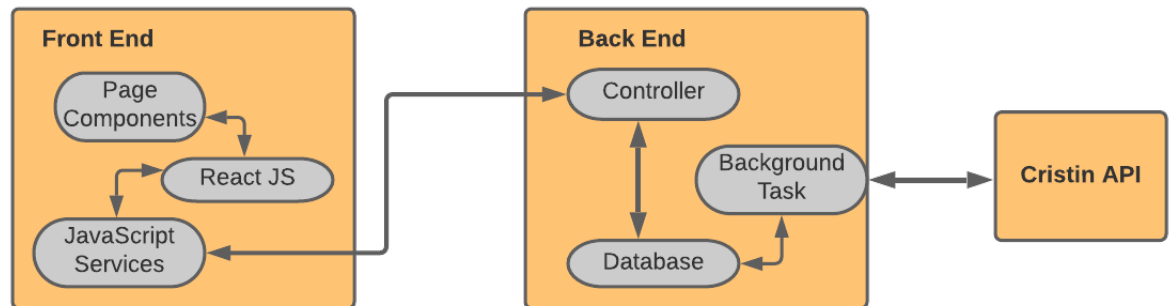


Figure 3: Overview

4.2 Non-Functional Requirements

4.2.1 Modifiability

Modifiability is a quality attribute of an architecture that concerns itself with the changes that may occur to a system after its release. The UiA-Research-Dashboard has two components, the

user interface component and the server component. The client and server components can be considered as separate systems that exist independently and communicate over a network. Since they have different areas of responsibilities and they can communicate over a network, thus, changing the task of either component has no effect on the other.

The components are further divided into multiple modules, for example the server includes controller module, Background task module and data access module (database and data models) as illustrated in figure [3]. The controller and Background task are dependent on the database. Changing the database implementation has impact on the functionalities of both controller and background task modules since they are dependent on it. However, since the dependency is managed through the dependency injection mechanism [10] that comes along with the ASP.NET Core Framework[18], changes can be made only the database context but not in the other modules that depend on it.

4.2.2 Usability

Easy-to-use software is critical for a positive user experience. The Publication Dashboard's usability characteristics are like those of most Dashboards. The system facilitates user engagement and improves the user's satisfaction. Additionally, the user has the ability to perform tasks quickly and easily. We minimize user's error in order to optimize user satisfaction. This is achieved by presenting options in the form of checkboxes.

4.2.3 Performance

Publication Dashboard introduced caching to improve the response time of user interaction. A slight delay in response time can have a negative impact on the user's experience and comfort, which is why we must pay close attention to the user's needs. At the front-end, a single-page application was used, which means that each time a user interacts with the application, it does not render the entire page; instead, it renders only the portion requested by the user.

4.3 Database

4.3.1 Database design

Designing the database took place throughout the development period. During a group meeting, the team discusses and identifies a functionality or use-case. The designing stage starts following

the choice of functionality, and the developer responsible for the design starts working on it. Usually, database designing, database creation and loading to database by fetching from remote data sources are executed in parallel, by the responsible developer.

The database design focused on creating a model that suits the visual reporting functionalities of the dashboard. For example, the dashboard intends to display the number of publications per year, reported by the departments in the UiA. The designing process thus creates a database model that can store the necessary information to support this functionality.

Most of the time, MySQL workbench was used to design the database before beginning coding. However, some relations were simple enough to be created while coding at the integrated development interface.

4.3.2 Fetching Data and Loading To Database

The data that will live in the local database is requested from CRISTIN and some are extracted from the UiA's web page during the initial populating phase. The general sequence of actions employed to load data to the target database is fetch, map to target models and store to the database as shown in figure4(a) in the case where CRISTIN is the data source and fig 4(b) for extracting from UiA.no.

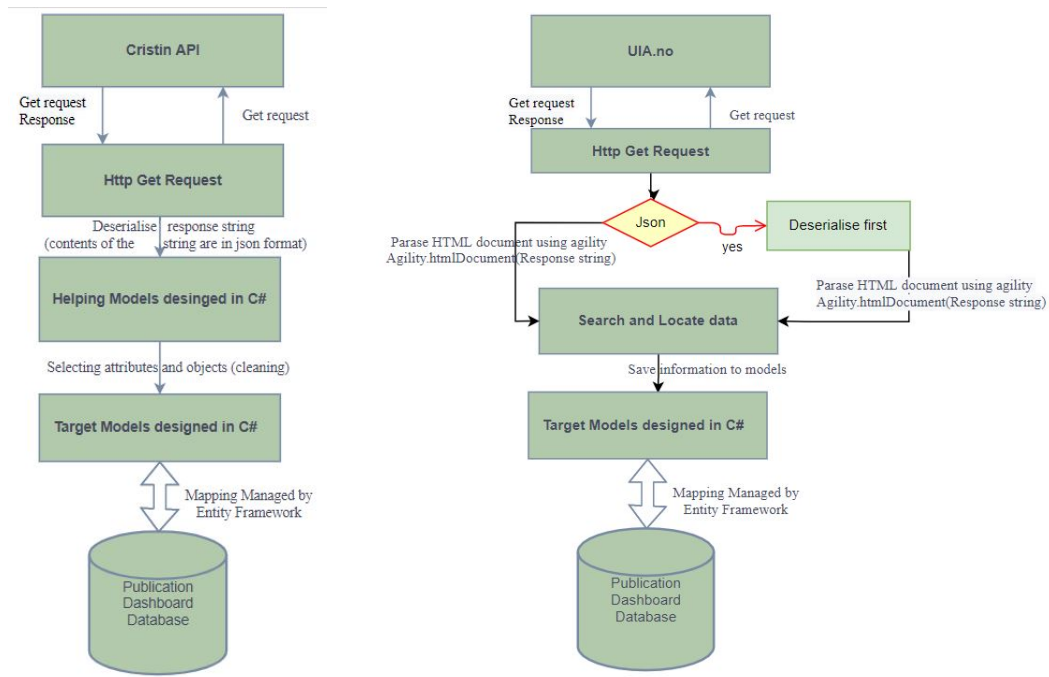
CRISTIN provides structured data in JSON³ format, via a REST API⁴, making it easier to select objects and/or attributes. UiA's web page, in comparison to CRISTIN's, does not have a convenient way to access the data available on the page. As a result, searching for and finding the required information is performed manually. Before extracting information from the UiA's web page, the page is downloaded and the HTML content loaded into a HTML parser extension known as HtmlAgilityPack⁵[27].

³JavaScript Object Notation

⁴Application Programming Interface

⁵.NET core library that aid in HTML parsing

```
var htmlDocument = new HtmlDocument();
htmlDocument.LoadHtml(htmlString);
```



(a) Simple graphical view of the (b) Simple graphical view of parse, located, extract and store operations
fetch, map to target model and
store operations

Figure 4: Simple graphical presentation of Data Fetch and Load to database

The data queried from CRISTIN has redundant data, it does not directly fit into the target models. To simplify the mapping process, helping models are prepared for the purpose of deserializing the data received in Java script object notation format. From the helping models the target models are created by cleaning the irrelevant data and then saved to database.

Entity Framework Core (EF) Managed Mapping

As mentioned in 1.4.5 entity framework core is Object-relational mapper. EF core manages the conversion between the object-oriented models to the data types of the target database and vice-versa as shown in fig 5. In this project C# is used as the server-side programming language , and SQLite as the development database. Entity framework thus converts between the data types of this two systems during storage and retrieval.

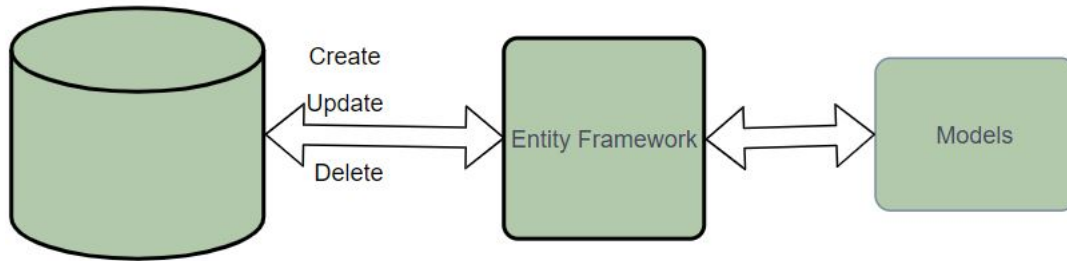


Figure 5: Entity Framework managed mapping (ORM)

4.3.3 Database Synchronisation With Remote Database(CRISTIN)

For the visual reporting to be reliable, latest information must be available at the database. For this reason the systems database has to be refreshed periodically. The system is configured to start a background task every 30 days to synchronise the database with the source database(CRISTIN). Therefore the database does not really store real-time information, and the research activities reported during the last synchronisation date are added to the database at the next update. The periodic data synchronisation can be depicted by the simple graph in figure



Figure 6: Synchronisation With Remote Database

. To save the last synchronisation date, which is the year of publication, a one-row table is used. The database is refreshed once a month, but because the synchronisation date is the year of publication, some publications will be retrieved multiple times but registered only once in the database.

4.3.4 Background tasks

The component of the application that implements the initial populating of the database and then the synchronisation of the database with the remote source is the Background service [10]. This module contains four classes: BackgroundService.sc, ServiceTask.cs, UrlHelper.cs, and Utilities.cs.

The class BackgroundService.sc is a service and is registered in the configuration class of the application. The Application runs this service periodically on a scheduled date. The class ServiceTask.cs contains the logic that sends requests, handles the response and saves to the database. BackgroundService.sc derives from the class ServiceTask.cs and thus has access to the operations in ServiceTask.cs. The other two classes UrlHelper.cs and Utilities.cs are used by the ServicesTask.cs

for example the `UrlHelper.cs` is used in preparing URL⁶ or requests to be sent to CRISTIN.

4.4 Frontend Architecture

4.4.1 Components

This project consists of several react component classes that are building block of the website. The institute component is a class that displays a chart for the different institutes in the University of Agder with regards to the amount of publication they have accomplished. The Faculty component is a component class that displays a chart for different faculties found in the UiA with regards to the publication they have published in the past few years. Additionally, it includes a checkbox functionality that enables users to select from among the various faculties at UiA and view a chart of institutes and the number of publications they have produced. Publication component displays a chart of the different publications made in the university in the past few years. Other components such as presentation video, which contains a quick tutorial video of the website.

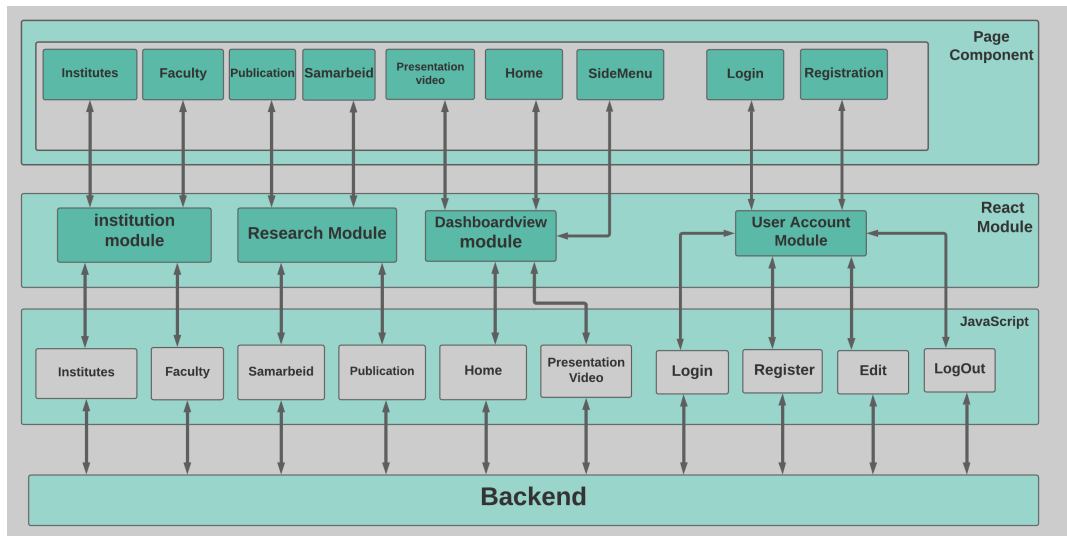


Figure 7: Component architecture

As it is shown in the figure above, all the page components are mapped to their respective react Module, which lets the application change its state. These components calls the JavaScript

⁶Uniform Resource Locator

services to retrieve the necessary data from the back-end, by using Axios and Fetch API. Once the JavaScript services get response from the backend, the new state will be sent back to the react modules and will be displayed in the Page components. Each react modules are directories that contains components with similar functionalities. There are five different react modules in this project. Institution module, which contains institutes and faculty components. Researches module contains publication, collaboration and GroupVsPublication components. DashboardView module contains the home page, privacy policy, sidemenu and the presentation video components. User Account module contains Register, Login/Logout, Delete and Edit components.

4.5 Testing

4.5.1 Postman - API-backend tests

At the beginning of the project, we have agreed to use Postman for testing purpose, when the back-end was being developed. Postman is a tool for automating various forms of unit testing. In our case, tests were made during development phase in order to verify that the new version of code is working precisely and did not break the application. If the program does break, postman makes it simple to fix the code before pushing it. Postman test can be sectioned according to their use. There are two types of tests: pre-request script section and test section. The pre-request section would execute before request was made, while test section executes after response has arrived.

RUN SUMMARY			1
▶	POST	Login	1 0
▶	POST	user registration	2 0
▶	POST	user eidt	1 0
▶	GET	List of Publication and institutes	1 0
▶	GET	Group research vs publication	1 0
▶	GET	Total Publication UIA	1 0
▶	GET	List Name of Faculties	1 0
▶	GET	Faculties and their publications based on ...	1 0
▶	GET	samarbeid	1 0

Figure 8: Automicated testing

4.5.2 Testing Amount of Publication Per Institutes

Publication are fetched from CRISTIN per institute. If the query result is large CRISTIN sends them in chunks of data that divides query result into multiple pages, which means sending multiple GET requests to find each page.

In April,19, 2021, a test was performed to check if the system fetches all publications per institute, to compare the amount saved in the local database and the total amount sent from CRISTIN. The total amount sent from CRISTIN can be checked from the field **X-Total-Count** at the response headers of the request. The test Performed shows the total amount fetched was correct, except for the institutes with **CRISTINUnitId: 201.18.1.0**, which the system registered 565 publication, against the 566 publication sent, see the table at Fig 13. Documentation for this test is available in Appendix E and at the url: <https://documenter.getpostman.com/view/11263066/TzJybFCB> which shows the header as illustrated in fig9.

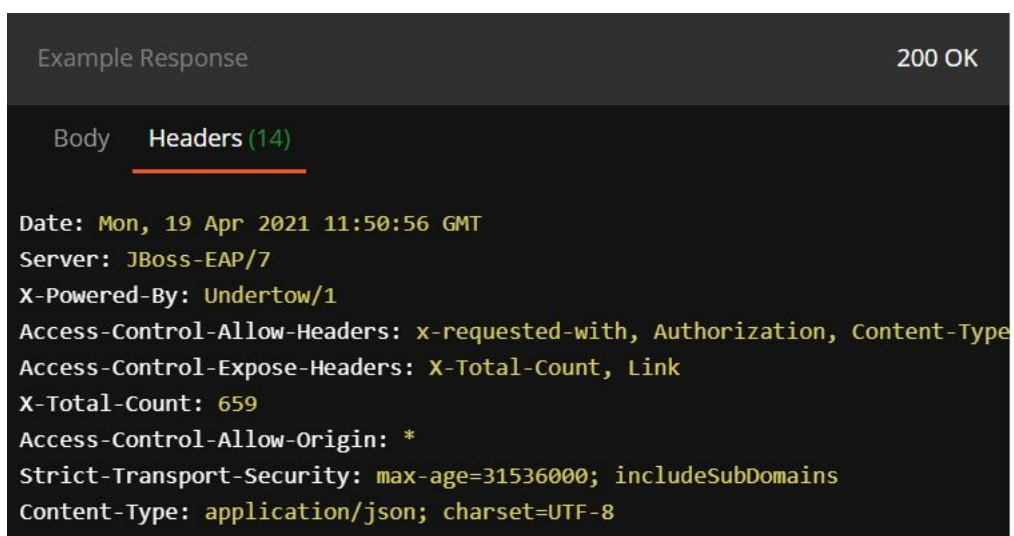


Figure 9: Illustration of Headers In Post Man

5 Result

5.1 High level System Architecture

Figure 9, displays an overview of the entire system. View is the main interface that the user is able to see and interact with. The controller, which updates the view, manages all interaction in the system. The controller interacts with the model in order to receive data. It is also responsible for updating the view based on the user's requests. The model is responsible for interacting with the database and return responses to all requests made by the controller, back to the controller. The background service is responsible for retrieving data from CRISTIN database and store it in the local database. It also maintains the database, meaning, it can update and edit the data in the database.

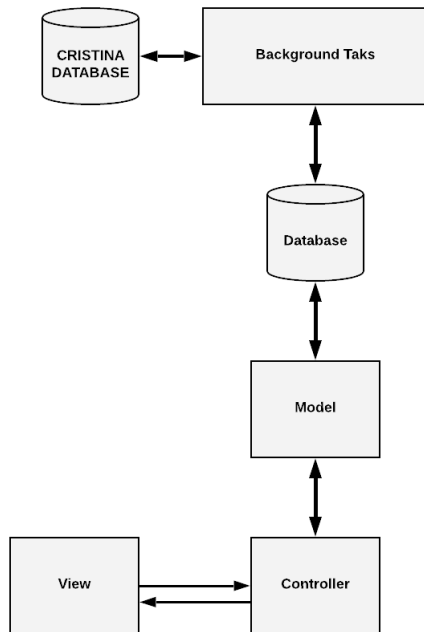


Figure 10: System Architecture

5.2 Database

The database of the system attempted to capture and model the entities that are involved in academic research activities and any relationships that may exist between them. This mainly revolve around entities research Institution, researcher, research activity and the various forms of relationship that exist between them.

These three represent high level concepts, which the modeling process started with and later expanded to include other entities. For example entities that are added to model the internal structure of UiA, because UiA have multiple faculties that can also have multiple departments. Other tables represent associations created from the many-to-many relationships of entities, for example the UiA cooperated with different national and international institutions in different publications, through the contributors to the research. But contributors in their part are affiliated to one or multiple research groups or institution during a particular period. Thus association tables are born out of such complex relationships.

The database is designed to suit Use-cases of the publication dashboard, like reporting the amount of research by department, or research groups. The database evolved as required to support new use-cases of the dashboard. Some of the use-case needed just modifying the existing entities, While others required the creation of new entities to incorporate capabilities that support this recent use-case. The final logical model of the database without authentication and authorisation mechanism is shown in fig [12]. The diagram in fig 12 is designed in MySQL Workbench, and it does not show inheritance relationships which are actually implemented at the end of the project period. The database diagram which presents more detailed information is generated from JetBrains Rider[8], which is the development platform we used, as shown in figure [11].

To add authentication and authorisation capability to the database, the entity model class **CurrentReGroupMember** inherited from **IdentityUser<Tkey>** class that lives at the **namespace⁷ Microsoft.AspNetCore.Identity** [22] and the table **AspNetUsers** is born to represent the **CurrentReGroupMember** entity at the physical database, as depicted at [11].

⁷Namespace: a way of organising programs into related elements [24]

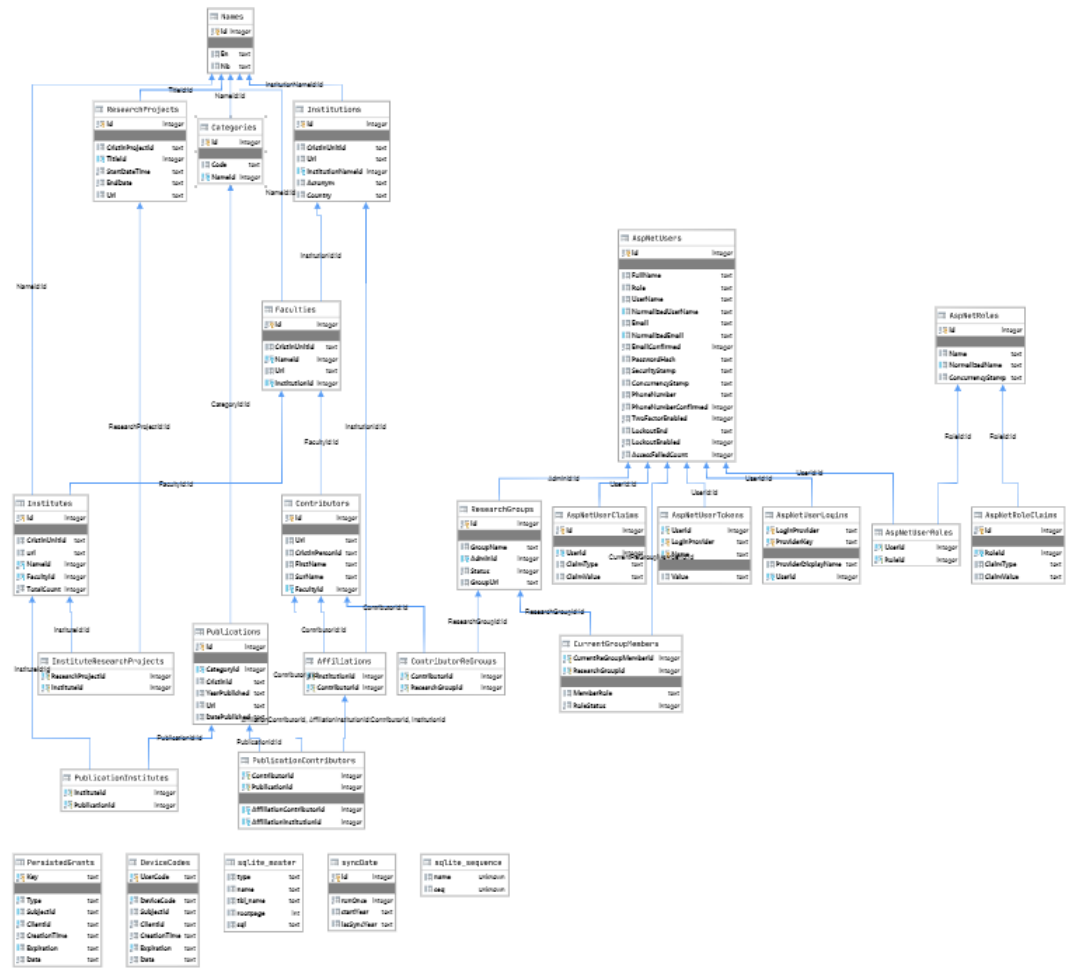


Figure 11: Database for Dashboard

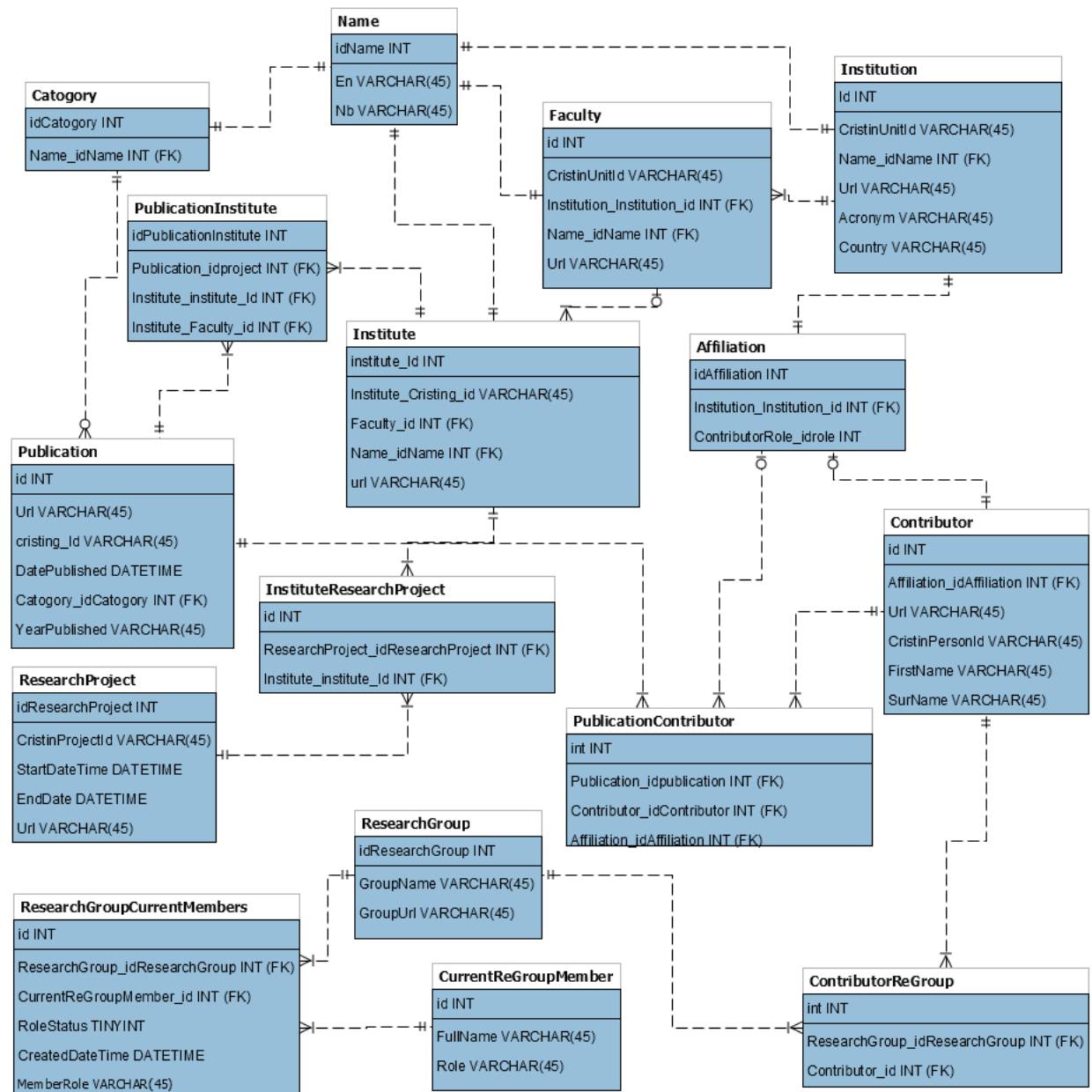


Figure 12: Database Designed in MySQL Workbench

5.3 Initially Populating the Database

The database is now ready to be loaded after it has been designed and created. When the background service (a service responsible for requesting the remote data sources) is launched for the first time. It configures the database and loads it with existing data fetched from the remote data sources. The initial database populating task prepares the database infrastructure by populating basis tables that will be referenced by their Id's when in other tables if required.

One important table that represents the core functionality of the system is ***Institutes*** table. The *Institutes* table is populated with the institutes of the University of Agder that included some attributes like name and publications published from 2017 until 2021. The result table looks as shown in fig 13. However, according to the university's organizational structure, an institute is assigned to a specific faculty, and one faculty may include multiple institutes. The **FacultyId** identifies the faculty to which the institute belongs. As a result, for an institute to be registered, the parent faculty must already exist and the relationship must be established by registering the **FacultyId** at the ***Institutes*** table. Thus, the initial loading of the database lays the groundwork for establishing such relationships and associations.

Id	CristinUnitId	url	NameId	FacultyId	TotalCount
1	201.14.1.0	https://api.cristin.no/v2/units/201.14.1.0	11	4	659
2	201.14.2.0	https://api.cristin.no/v2/units/201.14.2.0	12	4	1012
3	201.14.3.0	https://api.cristin.no/v2/units/201.14.3.0	13	4	646
4	201.14.4.0	https://api.cristin.no/v2/units/201.14.4.0	14	4	1149
5	201.15.1.0	https://api.cristin.no/v2/units/201.15.1.0	15	5	485
6	201.15.2.0	https://api.cristin.no/v2/units/201.15.2.0	16	5	336
7	201.15.3.0	https://api.cristin.no/v2/units/201.15.3.0	17	5	1177
8	201.15.4.0	https://api.cristin.no/v2/units/201.15.4.0	18	5	1132
9	201.17.1.0	https://api.cristin.no/v2/units/201.17.1.0	19	6	179
10	201.17.2.0	https://api.cristin.no/v2/units/201.17.2.0	20	6	382
11	201.17.3.0	https://api.cristin.no/v2/units/201.17.3.0	21	6	325
12	201.18.1.0	https://api.cristin.no/v2/units/201.18.1.0	22	7	565
13	201.18.3.0	https://api.cristin.no/v2/units/201.18.3.0	23	7	16
14	201.18.4.0	https://api.cristin.no/v2/units/201.18.4.0	24	7	539
15	201.18.6.0	https://api.cristin.no/v2/units/201.18.6.0	25	7	418
16	201.18.7.0	https://api.cristin.no/v2/units/201.18.7.0	26	7	372
17	201.19.1.0	https://api.cristin.no/v2/units/201.19.1.0	27	8	403
18	201.19.2.0	https://api.cristin.no/v2/units/201.19.2.0	28	8	393
19	201.19.3.0	https://api.cristin.no/v2/units/201.19.3.0	29	8	262
20	201.19.4.0	https://api.cristin.no/v2/units/201.19.4.0	30	8	354
21	201.20.1.0	https://api.cristin.no/v2/units/201.20.1.0	31	9	322
22	201.20.2.0	https://api.cristin.no/v2/units/201.20.2.0	32	9	74
23	201.20.3.0	https://api.cristin.no/v2/units/201.20.3.0	33	9	524
24	201.20.4.0	https://api.cristin.no/v2/units/201.20.4.0	34	9	486

Figure 13: Publication Per Institutes Since 2017:- table populated 19 April 2021

5.3.1 Caching

The importance of caching was clearly understood from the beginning of the project. The benefit of caching is to reduce the time needed to retrieve the data from the database. It improves the performance and scalability of the system. Hence, it is suitable for this project as the data we fetch changes infrequently. Fetching data repeatedly is very expensive and time consuming. Caching is a technique for storing frequently used data in memory [7]. When the same data is needed next time, it directly retrieves from the memory instead of the database. The most time consuming data in our system has been cached for instant access by the user. When a user requests data that is not already in the cache, it will retrieve it from the database and return it to the user, otherwise, it will retrieve it from the cache.

5.4 Application

5.4.1 Home page

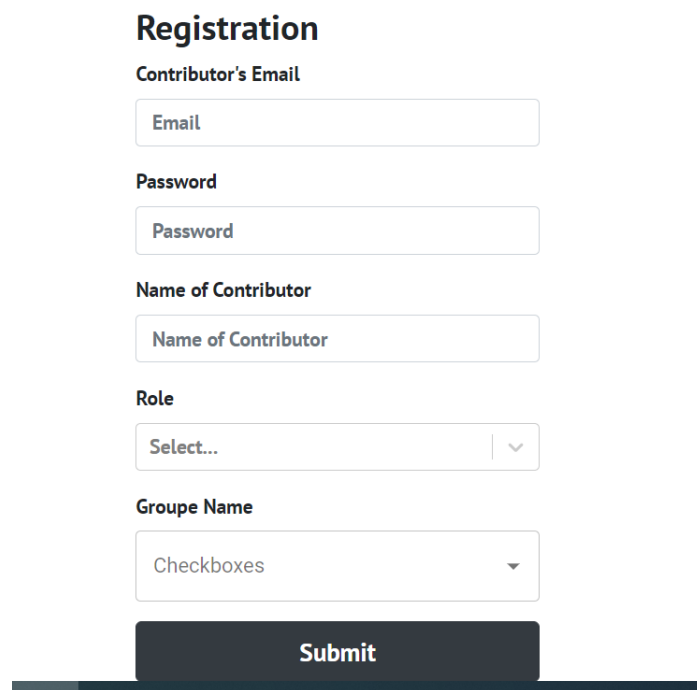
The first page a user come across when the website loads is the home page. The home page contains a complete list of all publications made in the University of Agder in the past few years. On the dashboard, there is a side menu, which contains different options and the user can click on each options, which gives him/her, access to all graphs and charts. A user does not need to register or log in to see the charts and graphs.



Figure 14: Home page

5.4.2 Registration

Super Admin is the administrator, who has the right to register new members, edit or delete existing members from research groups. To register a user, the super admin must first authenticate or login to the website. The Super admin is the only person that has the authority to create new accounts. The registration form has numerous fields, including roles, which indicates whether the user is an administrator or a user.



The image shows a registration form titled "Registration". It contains the following fields and controls:

- Contributor's Email:** A text input field with the placeholder text "Email".
- Password:** A text input field with the placeholder text "Password".
- Name of Contributor:** A text input field with the placeholder text "Name of Contributor".
- Role:** A dropdown menu with the placeholder text "Select..." and a downward arrow.
- Groupe Name:** A dropdown menu with the placeholder text "Checkboxes" and a downward arrow.
- Submit:** A dark blue button with the text "Submit" in white.

Figure 15: Home page

5.4.3 Group Research

The Group Vs Publication component displays a chart that shows the amount of publication made by different research groups. The graph shows 10 research groups at a time. It also includes a button that allows the graph to display the next top 10 publications made by the groups. The button can be clicked until all the amount of publications done by the research groups are shown.

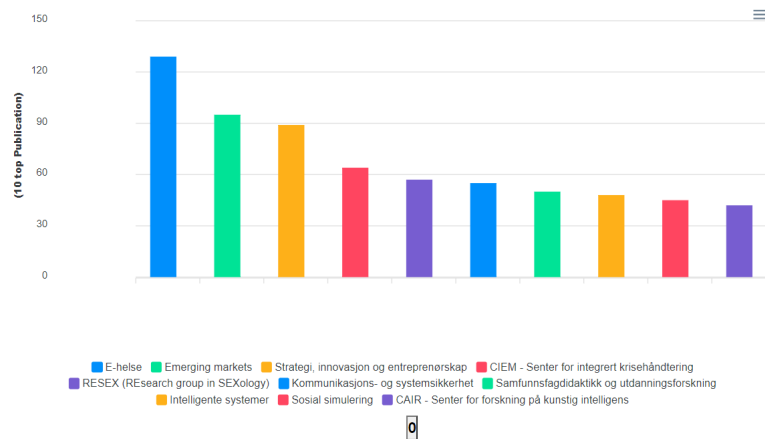


Figure 16: Top Ten Group Research

5.4.4 Faculties and institutes

The Faculty Component displays chart of the various faculties in University of Agder. In addition to this, it has a checkbox functionality, which allows users to choose between the different faculties. Once a user checked one or more faculties, the graph will change and display the different institutes under the chosen faculties.

6 Discussion

6.1 Results

The plan for this project was to build a tidy and responsive website for the University of Agder. This website uses data visualizations methods such as, charts to display research data retrieved from the CRISTIN database. This data is concerned with the researches, researchers and contributors, groups, amount of publication made, and other institutions UiA has collaborated with. This will be displayed on the view in a chart form to show the statistics.

The website we built offers most of the requirements set out for it. It can successfully fetch data from the CRISTIN API and display it to the users. It gives access to the admin to add new members or delete existing member of a certain research group. In addition to this, the website offers a quick tutorial of how the website works, to make it easier for the users to interact with it. However, there is still a space for it to be more developed and include more functionalities.

6.2 Process

The developmental process of this project had a quite good workflow. We followed a good plan and tried to stick with it. The group had a meeting several days a week, both physically and digitally. This helped us speed up the working process and makes the group stay organized. The group divides the workload amongst the members and every member had their own share of work. We wrote the report side-by-side while developing the product. Once we came close to finishing the developmental process of the product, we focused fully on writing the report. We had to redo some of the coding as we had issues of incompatibility with some of the libraries we had used previously. However, we managed to solve the problems and the website works perfectly. Once a week we had a meeting with our supervisor, which helped as gain different ideas and stay on track. Generally, the developmental process was satisfactory.

6.3 Challenges

We've encountered a number of difficulties while developing the front-end part of this project, including incompatible libraries. We experimented with various libraries capable of displaying charts. The majority were incompatible until we discovered the correct one. We didn't use any react templates, we started building the website from scratch, hence, it was quite challenging to

do self-taught in many aspects of this project. In addition to these, there were other challenges we encountered, for example, we were told to include additional functionalities later in the project which made the developing process somehow tight and stressful. However, we managed to come up with a solution.

6.3.1 Choosing best filter Parameters

One main challenge of the project was, devising a strategy to import the huge amount of research results data (CRISTIN calls it "result"). All the entries registered in CRISTIN by the University of Agder can be fetched by simply including the parameter **institution=UiA** in the request URL⁸ to the API's end-point **GET results?parameters**. However, such a strategy would have made it challenging to make connections like the faculty or institute participated in a research, which is particularly important for the aggregation task of the system. After considering different strategies, the search for strategy settled on retrieving the data per institutes. This may not be the best solution, giving that there are over twenty institutes, and request must be sent separately for each institute.

6.3.2 Authentication and Authorisation at the database level

Authentication is a mechanism a system employs to confirm a client is who a client claim to be. Authorisation is giving privileges and permissions to a user to perform specific actions.

A takeaway from this project is that Adding authentication and authorisation to a web application with react as the client app, should be planned at the first stage of the development. Once the database is designed, created and configured, it became costly and challenging to add authentication and authorisation capabilities to the database.

The main reason this was not planned at the beginning is that, it was assumed such a feature was not needed (as implied by the customer). However, when the product came near to completion (with the planned features), a request to add authentication and authorisation to the system was requested, and the group agreed to give it a try.

The attempt to implement this functionality involved modifying the database, to make room for the registration of a user with attributes that will be used for authentication and authorisation purposes. Modifying the database and configuring it succeeded.

⁸Universal Resource Locator

from the point of view of the process, this can be considered a failure to anticipate change and implement it when requested. The choice of the incremental method was exactly for such purposes. But the limited time we have also played a major role, as this is happening at the end of the planned project implementation period and the attempt had to be done in a hurry, because it took time that was supposed to be for report writing and deployment.

6.4 Deployment challenge

The software was to be deployed to a virtual machine at the University of Agder, as a docker container. However, Even though the application started as expected, accessing the web was a problem, a reason we suspected is a technical issue that we do not have control over at the Virtual Machine.

6.5 Future Work

At the moment, this web application works only for University of Agder. A future development of this web application would be enabling it to function for more than one university. All universities will be able to show their own research activities using this application. Currently, it works on all platforms such as, Windows, Mac OS or Linux on a computer. However, It is also possible to build it in a mobile application form. The website currently offers charts in the form of bar chart and pie chart. In the future, we can give users options to choose from different types of graphs, tables, trees and charts. Additionally, the charts do not describe details of a research. For example, it might display a graph that shows a faculty and the publications they made but it doesn't state the details of those publications such as, the names of the contributors, and other details. Thus, in the future we can be able to display a detailed information about a certain research in addition to the charts.

7 Conclusion

The product vision for this project was to develop a website that displays all research publication accomplishments done in the University of Agder.

The group faced a challenge due to the current situation of the corona virus as we had to depend on our own researching skills to learn different things without much of help from other people, in order to develop the website. However, with all the ups and downs, we manage to develop a website that successfully presents an organized charts that display the researches published by each faculties, institutes, research groups and institutions UiA collaborated with in the past few years. The website also allows authentication and authorization. The super administrator is allowed to add, delete and update the status of a user from a research group.

This helps UiA to control the accomplishments the university has achieved throughout the years. The website have a full ability to be developed further and also other institutions and companies will have the opportunity to be able to use it. We have used most of the knowledge we were taught during our stay in the university and have worked more professionally than we ever did in any other projects. In conclusion, the tools utilized in the development of this product were quite adequate. However, if tools such as Cypress, a front-end automated testing tool, had been used during the developmental phase, rather than manually testing the front-end, we would have saved time and increased our productivity. The method used to develop this product was quite satisfactory, and it aided us in resolving issues that arose throughout the development process. As a group, we are pleased with the final product and the experience gained while working on it.

References

- [1] *.NET documentation | Microsoft Docs*. <https://docs.microsoft.com/en-us/dotnet/>. (Accessed on 02/20/2021).
- [2] *“Promise”*. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise. (Accessed on 04/20/2021). Jan. 2018.
- [3] *1253983136805.pdf*. <https://www.forskningsradet.no/siteassets/publikasjoner/1253983136805.pdf>. (Accessed on 02/15/2021).
- [4] Puder Arno, Römer Kay, and Pilhofer Frank. *Distributed Systems Architecture : A Middle-ware Approach*. The MK/OMG Press. Morgan Kaufmann, 2006. ISBN: 9781558606487. URL: <http://search.ebscohost.com/login.aspx?direct=true&db=e000xww&AN=166194&site=ehost-live>.
- [5] *AURA - Agder University Research Archive - Universitetet i Agder*. <https://www.uia.no/bibliotek/forskning-og-publisering2/aura-agder-university-research-archive>. (Accessed on 05/08/2021).
- [6] *Brage - åpent vitenarkiv | Unit*. <https://www.unit.no/tjenester/brage-åpent-vitenarkiv>. (Accessed on 05/10/2021).
- [7] *Cache in-memory in ASP.NET Core | Microsoft Docs*. <https://docs.microsoft.com/en-us/aspnet/core/performance/caching/memory?view=aspnetcore-5.0>. (Accessed on 04/20/2021).
- [8] D. Comartin. *The Cross-Platform .NET IDE from JetBrains*. <https://www.jetbrains.com/rider/>. (Accessed on 02/15/2021).
- [9] *Data Visualization Technology | Transforming Data with Intelligence*. <https://tdwi.org/articles/2011/11/09/research-excerpt-data-visualization-technology.aspx>. (Accessed on 02/25/2021).
- [10] *Dependency injection in .NET | Microsoft Docs*. <https://docs.microsoft.com/en-us/dotnet/core/extensions/dependency-injection>. (Accessed on 04/29/2021).
- [11] *digital science*, <https://www.digital-science.com/products/overleaf/>. (Accessed on 02/15/2021).

- [12] *Entity Framework / Microsoft Docs*. <https://docs.microsoft.com/en-us/aspnet/entity-framework>. (Accessed on 02/20/2021).
- [13] *Fielding Dissertation: CHAPTER 5: Representational State Transfer (REST)*. https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm. (Accessed on 05/09/2021).
- [14] Ben Fry. *Visualizing data*. eng. Beijing: O'Reilly, 2007. ISBN: 0596514557.
- [15] *GraphQL / A query language for your API*. <https://graphql.org/>. (Accessed on 02/20/2021).
- [16] P. Halliday. "How To Use Axios with React," <https://www.digitalocean.com/community/tutorials/react-axios-react>. (Accessed on 04/20/2021). Aug. 2020.
- [17] *How to use Postman for API Testing Automation / BlazeMeter*. <https://www.blazemeter.com/blog/how-use-postman-manage-and-execute-your-apis>. (Accessed on 05/10/2021).
- [18] *Introduction to ASP.NET Core / Microsoft Docs*. <https://docs.microsoft.com/en-us/aspnet/core/introduction-to-aspnet-core?view=aspnetcore-5.0>. (Accessed on 04/29/2021).
- [19] *Is Single Page Application (SPA) Development Worth in 2021?* <https://aglowiditsolutions.com/blog/why-single-page-application-development/>. (Accessed on 05/10/2021).
- [20] *Jira / Issue & Project Tracking Software / Atlassian*. <https://www.atlassian.com/software/jira>. (Accessed on 03/23/2021).
- [21] *JSON Web Token Introduction - jwt.io*. <https://jwt.io/introduction>. (Accessed on 05/10/2021).
- [22] *Microsoft.AspNetCore.Identity Namespace / Microsoft Docs*. <https://docs.microsoft.com/en-us/dotnet/api/microsoft.aspnetcore.identity?view=aspnetcore-5.0>. (Accessed on 05/01/2021).
- [23] *MySQL :: MySQL Workbench*. <https://www.mysql.com/products/workbench/>. (Accessed on 04/15/2021).
- [24] *Namespaces - C# language specification / Microsoft Docs*. <https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification/namespaces>. (Accessed on 05/01/2021).
- [25] *Om Cristin - Cristin*. <https://www.cristin.no/om-cristin/>. (Accessed on 01/26/2021).

- [26] Michael P. Papazoglou. *WEB services : principles and technology*. eng. Person Prentice Hall, 2008. ISBN: 9780321155559.
- [27] *parser / Html Agility Pack*. <https://html-agility-pack.net/parser>. (Accessed on 04/15/2021).
- [28] *Publisere innførsler fra Cristin*. <https://dok.unit.no/brage/veiledninger/publisere-innforsler-fra-cristin>. (Accessed on 05/10/2021).
- [29] *React – A JavaScript library for building user interfaces*, <https://reactjs.org/>. (Accessed on 05/10/2021).
- [30] Abraham Silberschatz. *Database system concepts*. eng. Seventh edition. New York: McGraw Hill Education, 2019. ISBN: 9780078022159.
- [31] *Simple Object Access Protocol (SOAP) 1.1*. <https://www.w3.org/TR/2000/NOTE-SOAP-20000508/>. (Accessed on 05/09/2021).
- [32] Ian Sommerville. *Software engineering*. eng. Boston Mass., 2016.
- [33] *SQL - Wikipedia*. https://en.wikipedia.org/wiki/SQL{#}cite_note-chamberlin2001-5. (Accessed on 04/15/2021).
- [34] *SQLite Home Page*. <https://sqlite.org/index.html>. (Accessed on 04/20/2021).
- [35] *Technical Specifications - DSpace*. <https://duraspace.org/dspace/resources/technical-specifications/>. (Accessed on 05/10/2021).
- [36] *The Cristin system - PhD on track*. <https://www.phdontrack.net/share-and-publish-the-cristin-system/>. (Accessed on 02/15/2021).
- [37] *The GraphQL Foundation | An open and neutral home for the GraphQL community*. <https://foundation.graphql.org/>. (Accessed on 02/20/2021).
- [38] *Tutorial: Intro to React – React*. <https://reactjs.org/tutorial/tutorial.html>. (Accessed on 03/23/2021).
- [39] *Veiledninger*. <https://dok.unit.no/brage/veiledninger>. (Accessed on 05/09/2021).
- [40] *Vitenskapelig publisering*. <https://www.forskningsradet.no/indikatorrapporten/indikatorrapporten-dokument/vitenskapelig-publisering/>. (Accessed on 02/16/2021).

- [41] Mike Wasson. *Single page application: KnockoutJS Template*. <https://docs.microsoft.com/en-us/aspnet/single-page-application/overview/introduction/knockoutjs-template/>. (Accessed on 01/26/2021).
- [42] *What is Bitbucket? | Evaluator Resources | Atlassian Documentation, 2021*). <https://confluence.atlassian.com/confeval/development-tools-evaluator-resources/bitbucket/bitbucket-what-is-bitbucket>. (Accessed on 02/15/2021).

Appendices

All appendices are separate files except Appendix A.

A Group Declaration

Group Declaration

B Group Contract

Group Contract

C Group Meeting Discussion

Group Meeting Discussion

D Preliminary Project

Preliminary Project

E Querying CRISTIN for publication from PostMan

Postman CRISTIN

F Dashboard Database

Dashboard Database

G Publication per institute

Publication per institute

Appendix A

Obligatorisk gruppeerklæring

Den enkelte student er selv ansvarlig for å sette seg inn i hva som er lovlige hjelpemidler, retningslinjer for bruk av disse og regler om kildebruk. Erklæringen skal bevisstgjøre studentene på deres ansvar og hvilke konsekvenser fusk kan medføre. Manglende erklæring fritar ikke studentene fra sitt ansvar.

1.	Vi erklærer herved at vår besvarelse er vårt eget arbeid, og at vi ikke har brukt andre kilder eller har mottatt annen hjelp enn det som er nevnt i besvarelsen.	Ja
2.	Vi erklærer videre at denne besvarelsen: • Ikke har vært brukt til annen eksamen ved annen avdeling/universitet/høgskole innenlands eller utenlands. • Ikke refererer til andres arbeid uten at det er oppgitt. • Ikke refererer til eget tidligere arbeid uten at det er oppgitt. • Har alle referansene oppgitt i litteraturlisten. • Ikke er en kopi, duplikat eller avskrift av andres arbeid eller besvarelse.	Ja
3.	Vi er kjent med at brudd på ovennevnte er å betrakte som fusk og kan medføre annullering av eksamen og utestengelse fra universiteter og høyskoler i Norge, jf. Universitets- og høyskoleloven §§4-7 og 4-8 og Forskrift om eksamen §§ 31.	Ja
4.	Vi er kjent med at alle innleverte oppgaver kan bli plagiatkontrollert.	Ja
5.	Vi er kjent med at Universitetet i Agder vil behandle alle saker hvor det forligger mistanke om fusk etter høyskolens retningslinjer for behandling av saker om fusk.	Ja
6.	Vi har satt oss inn i regler og retningslinjer i bruk av kilder og referanser på biblioteket sine nettsider.	Ja
7.	Vi har i flertall blitt enige om at innsatsen innad i gruppen er merkbart forskjellig og ønsker dermed å vurderes individuelt. Ordinært vurderes alle deltakere i prosjektet samlet.	Nei

Publiseringsavtale

Fullmakt til elektronisk publisering av oppgaven Forfatter(ne) har opphavsrett til oppgaven. Det betyr blant annet enerett til å gjøre verket tilgjengelig for allmennheten (Åndsverkloven. §2).

Oppgaver som er unntatt offentlighet eller taushetsbelagt/konfidensiell vil ikke bli publisert.

Vi gir herved Universitetet i Agder en vederlagsfri rett til å gjøre oppgaven tilgjengelig for elektronisk publisering:	Nei
Er oppgaven båndlagt (konfidensiell)?	Nei
Er oppgaven unntatt offentlighet?	Nei